2 Sets and natural numbers

2.1 Operations on sets

Let the sets $A = \{1, 3, 5, 7, 9\}$, $B = \{2, 4, 6, 8, 10\}$ and $D = \{5, 6, 7, 8, 9, 10\}$.

Explicitly write down the sets described by the following expressions:

a)	$A \cup B$,	b)	$A \cap B$,	c)	$A \setminus B$,
d)	$A \setminus D$,	e)	$B \setminus D$,	f)	$D \setminus A$,
g)	$D \setminus B$,	h)	$D \setminus (A \cup B)$,	i)	$D \setminus (A \cap B)$.

2.2 Cardinality of finite sets

Determine the number of elements of the following sets:

a) $\{1, 4, 6\}$, b) $\varnothing$, c) $\{\varnothing\}$, d) $\{\varnothing, \{1, 2, 3\}\}$.

2.3 Sets of sets of ...

Let $M = \{\{1,2,3,5\},\{3,2,4,1\},\{4,3,1\}\}$.

1. Determine $\bigcap M$ and $\bigcup M$.

   Reminder: $\bigcap M$ and $\bigcup M$ denote the result of taking the intersection and union of sets which are elements of M, respectively. E.g. $\bigcap M = \{1,2,3,5\} \cap \{3,2,4,1\} \cap \{4,3,1\}$.

2. Determine $N = \{x \in 2^{\bigcup M} \mid \text{for every } m \in M \text{ we have } x \subseteq m\}$.

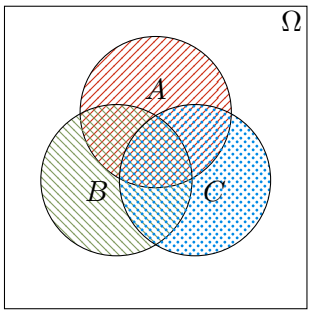
   Reminder: 2^X denotes the power set of X, i.e. the set whose elements are exactly all subsets of X (i.e.: $Y \in 2^X$ iff $Y \subseteq X$).

3. Determine $\bigcup N$.

2.4 Venn-Diagrams

Relations between sets can be visualized using Venn diagrams.

For this the sets are drawn as circles or ellipses, that overlap so that every possibility for an object to be an element of the given sets is represented. For three sets $A, B, C \subseteq \Omega$ the diagram looks as follows:



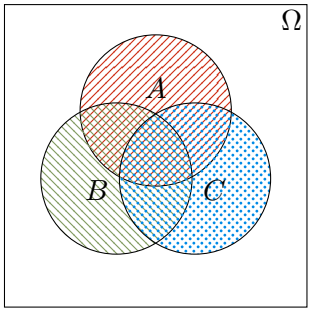
Determine three sets A, B, C with as few elements as possible using Venn diagrams so all of the following constraints are satisfied at once.

(1) $A \cap B \cap C = \varnothing$ (2) $A \cap B \neq \varnothing$
(3) $B \cap C \neq \varnothing$ (4) $A \cap C \neq \varnothing$

2.5 Simplifying set expressions

We fix a set Ω as our universe. Let A, B, C be subsets of Ω.

The general Venn diagram is then



Each of the eight regions of the Venn diagram corresponds to an intersection of the given sets A, B, C or their complements $\overline{A}, \overline{B}, \overline{C}$ (wrt. the fixed Ω).

Hence, no matter how complicated a set expression built from A, B, C might be, it can be simplified to a union of those 8 basic intersections.

E.g. the expression $(A \setminus (B \cup C)) \cup ((B \cap C) \setminus A)$ describes the same set as $(A \cap \overline{B} \cap \overline{C}) \cup (\overline{A} \cap B \cap C)$.

Similar express the following set expressions as union of those eight basic intersection that correspond to the eight regions of the Venn diagram.

1. $((A \cup (B \cup C)) \cap (C \setminus A))$.
2. $\overline{(A \cap \overline{B \cap C})} \setminus (A \setminus B)$.

2.6 Set difference and set complement

We fix a set Ω as a base set (universe).

Let $M, N \subseteq \Omega$ be arbitrary subsets of Ω. Argue, that:

$$M \setminus N = M \cap \overline{N}$$

As a reminder, by definition we have

- $x \in M \setminus N$ IFF $(x \in M \text{ and } x \notin N)$
- $x \in M \cap \overline{N}$ IFF $(x \in M \text{ and } x \in \Omega \setminus N)$

Use these definitions in your argumentation.

The main purpose of this exercise is to memorize what it means to show that two sets are identical. You can use a Venn diagram to first motivate that the identity holds, but then write the argument down similar to the "bandersnatch example" in the slides.

2.7 The Inclusion-Exclusion Principal using Venn-Diagrams

For a set A one writes $|A|$ for the cardinality of the set. For finite sets, where one could (at least in theory) write down all elements by hand, $|A|$ is just the number of elements in that set.

1. From school you might already know that

   "$|A \cup B| = |A| + |B|$ IFF $A \cap B = \varnothing$"

   as in general "$|A| + |B|$" will count the elements in $A \cap B$ twice.

   In general, we therefore have $|A \cup B| = |A| + |B| - |A \cap B|$.

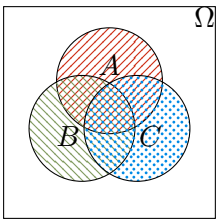
   Check this for $A = \{1, 2, 3\}$ and $B = \{2, 3, 4\}$.

2. Now consider the three sets

   $$A = \{2, 4, 6, \ldots, 30\} \qquad B = \{3, 6, 9, \ldots, 30\} \qquad C = \{5, 10, 15, \ldots, 30\}$$

   i.e. A (B, C) is the set of all positive multiples of 2 (3, 5) up to 30.

   Determine $|A \cup B \cup C|$. To this end, consider the general Venn diagram for three sets

   and think about why we have in general:

   $$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |A \cap C| + |A \cap B \cap C|$$

   One way of arguing that this formula is correct is to reduce each of the involved sets to an union of the eight basic intersections corresponding to the eight regions of the Venn diagram. E.g. $A = (A \cap \overline{B} \cap \overline{C}) \cup (A \cap B \cap \overline{C}) \cup (A \cap \overline{B} \cap C)$. As the basic intersections are disjoint, their cardinalities can simply be added.

   Another, more efficient way is to treat $(B \cup C)$ as a new set D so that $|A \cup D| = |A| + |D| - |A \cap D|$. Then it remains to determine $|D| = |B \cup C|$ and $|A \cap D| = |(A \cap B) \cup (A \cap C)|$.

3. (*) Draw a Venn diagram for four arbitrary sets $A, B, C, D \subseteq \Omega$ (or look it up on wikipedia; there you can also find the one for five arbitrary sets), then think about how to calculate $|A \cup B \cup C \cup D|$ only using the cardinalities of the intersections between A, B, C and D in a similar fashion.

   Hint: You can apply the same trick again by setting $E := B \cup C \cup D$ so that $|A \cap E| = |A| + |E| - |A \cap E|$.

4. (*) Determine the exact number of numbers between 1 and 1 000 000 that are divisible without remainder by at least one of the numbers 2, 3, 5 and 7 without writing a computer program (or counting ;-)).

2.8 Set of tuples

Let $\Omega = [6] = \{1, 2, 3, 4, 5, 6\}$ and $A = \{1, 2, 3\}$ and $B = \{2, 4, 6\}$.

To add to the confusion (that is a (bad) joke), we will use the complement notation both w.r.t. Ω and $\Omega \times \Omega$: If $X \subseteq \Omega$, then $\overline{X} = \Omega \setminus X$, but if $X \subseteq \Omega \times \Omega$, then $\overline{X} = (\Omega \times \Omega) \setminus X$.

Determine the following sets and their cardinality by explicitly listing their elements:

1. $A \times A$, $A \times \overline{A}$, $\overline{A} \times A$, $\overline{A} \times \overline{A}$, $\overline{A \times A}$.
2. $A \times (A \cup B)$, $(A \cup B) \times A$, $\overline{A \times (A \cup B)}$.
3. $A \times (A \times A)$, $(A \times A) \times A$, $A \times A \times A$.

2.9 Set of tuples and databases

A set $R \subseteq A_1 \times A_2 \times \ldots \times A_k$ is called a (k-ary) relation.

In computer science, relations are used as the basis of rrelational databases"which are a collection of (named) relations where each relation is usually represented as table.

For instance, consider the relation "IdToName" defined by the table

id	firstname	lastname
0	Alice	Rock
1	Bob	Paper
2	Eve	Scissors

and the relation "IdToJob"

id	job
0	Sender
1	Receiver
2	Eavesdropper

The tables are just a convenient way of representing the relations, each row simply stands for one tuple, i.e. we have

$$\begin{aligned} \text{IdToName} &= \{(0, \text{Alice}, \text{Rock}), (1, \text{Bob}, \text{Paper}), (2, \text{Eve}, \text{Scissors})\} \\ \text{IdToJob} &= \{(0, \text{Sender}), (1, \text{Receiver}), (2, \text{Eavesdropper})\} \end{aligned}$$

The elementary operations to work with databases are the standard set operations, and further (some variants) of the "join" and the "projection".

- The "join" $R_1 \bowtie_{i=j} R_2$ takes two relations R_1, R_2 and concatenates every tuple $t_1 \in R_1$ with every tuple $t_2 \in R_2$ if the i-th component of t_1 coincides with the j-th component of t_2.
- the "projection" $\pi_{i_1,\ldots,i_r}(R)$ takes a relation R and reduces every tuple $t \in R$ to tuple $(t_{i_1}, t_{i_2}, \ldots, t_{i_r})$.

Assuming that we index tuples starting at 1, e.g. $\text{IdToName} \bowtie_{1=1} \text{IdToJob}$ yields

$$\{(0, \text{Alice}, \text{Rock}, 0, \text{Sender}), (1, \text{Bob}, \text{Paper}, 1, \text{Receiver}), (2, \text{Eve}, \text{Scissors}, 2, \text{Eavesdropper})\}$$

We can use the projection to only keep e.g. the firstname and the job description:

$$\pi_{2,5}(\text{IdToName} \bowtie_{1=1} \text{IdToJob}) = \{(\text{Alice}, \text{Sender}), (\text{Bob}, \text{Receiver}), (\text{Eve}, \text{Eavesdropper})\}$$

In principle, binary relations, i.e. sets of pairs resp. tables with just two columns suffice. E.g. we can further "factorize" IdToName into IdToNameId, NameIdToFirstname, NameIdToLastname:

id	nameid
0	0
1	1
2	2

nameid	firstname
0	Alice
1	Bob
2	Eve

nameid	lastname
0	Rock
1	Paper
2	Scissors

1. Adapt the "query" $\pi_{2,5}(\text{IdToName} \bowtie_{1=1} \text{IdToJob})$ so that it still computes the same relation.
2. The main reason for "factoring" data into several relations (tables) is to avoid duplicated data.

   Assume Alice and Bob get married and (obviously!) Bob decides to change his lastname to "Rock".

   Can you further change the structure of the database so that every lastname is stored at most once?

Remark: This is the essence of SQL. A statement like

SELECT columns $i_1, \ldots, i_r$ FROM relation R WHERE property P has to be satisfied

stands for

$$\pi_{i_1,\ldots,i_r}(\{t \in R \mid t \text{ has property } P\})$$

To "join" the data (information) distributed over several relations (tables), SQL allows to state the names of multiple relations after FROM.

2.10 More on the basis of databases

Consider the music streaming service of your choice. They host a database which usually contains, among others, the following tables, which are sets of tuples of identical lengths (also referred to as 'relations'):

$$\begin{array}{lcl}
\text{AlbumDescr} & = & \{(\text{albumid}_i, \text{artistid}_i, \text{year}_i, \text{title}_i) \in \mathbb{N}_0 \times \mathbb{N}_0 \times \mathbb{N}_0 \times \mathsf{String} \mid \ldots\} \\
\text{TrackDescr} & = & \{(\text{trackid}_i, \text{title}_i, \text{albumid}_i) \in \mathbb{N}_0 \times \mathsf{String} \times \mathbb{N}_0 \mid \ldots\} \\
\text{ArtistDescr} & = & \{(\text{artistid}_i, \text{artistname}_i, \text{isperson}_i) \in \mathbb{N}_0 \times \mathsf{String} \times \{0,1\} \mid \ldots\} \\
\text{ArtistToBand} & = & \{(\text{artistid}_i, \text{artistid}_i) \in \mathbb{N}_0 \times \mathbb{N}_0 \mid \ldots\} \\
\text{GuestToTrack} & = & \{(\text{artistid}_i, \text{trackid}_i) \in \mathbb{N}_0 \times \mathbb{N}_0 \mid \ldots\}
\end{array}$$

A 'relational program' is a sequence of definitions of relations ('assignments') of the form '$X \coloneqq Y \otimes Z$', where Y, Z are identifiers for already defined relations (again, i.e. a set of tuples with same length), and $\diamond$ a placeholder for the typical set operations including the join $\bowtie$, which is a conditional concatenation of tuples with

$$R \bowtie_{i_0=j_0} S \coloneqq \{(r_1, \ldots, r_n, s_1, \ldots, s_m) \mid (r_i)_{i\in[n]} \in R, (s_j)_{j\in[m]} \in S, r_{i_0} = s_{j_0}\},$$

so $\diamond \in \{\cup, \cap, \setminus, \Delta, \bowtie\}$, or $X \coloneqq \pi_t(Y)$ where t is a tuple of indices and π is the projection operator (which only keeps certain indices of our tuples, with

$$\pi_{i_1,\ldots,i_k}(S) \coloneqq \{(s_{i_1}, \ldots, s_{i_k}) \mid (s_i)_{i\in[l]} \in S\}.$$

Further, the right-hand side of the assignment may also be a term including relations which have been calculated using the mentioned operations $\{\cup, \cap, \setminus, \Delta, \bowtie, \pi\}$.

1. Let $t \in \mathsf{String}$ be some track title. Determine the output relation Z of the following relational program:

   $U \coloneqq \text{TrackDescr} \bowtie_{2=1} \{(t)\}$

   $V \coloneqq \pi_1(U)$

   $W \coloneqq \pi_3(U)$

   $X \coloneqq \pi_1(\text{GuestToTrack} \bowtie_{2=1} V)$

   $Y \coloneqq \pi_2(\text{AlbumDescr} \bowtie_{1=1} W) \cup X$

   $Z \coloneqq \pi_2(\text{ArtistDesc} \bowtie_{1=1} Y)$ – Output.

2. Write a "relational program" that outputs the relation which contains all 2-tuples with artist names who are persons and track titles which feature the artist directly or as a guest.

   For comparison, write down a set expression which yields the same output – what is the advantage of the representation as a program versus the representation as a set expression?

3 Natural numbers: principle of induction

3.1 Closed expressions for sums

Use induction to show the following claims for all $n \in \mathbb{N}_0$:

1. $\sum_{i=0}^{n} i = \frac{n \cdot (n+1)}{2}$
2. $\sum_{i=0}^{n} i^2 = \frac{n \cdot (n+1) \cdot (2n+1)}{6}$
3. The n-th *Fibonacci numbers* F_n is defined inductively: $F_0 := 0$, $F_1 := 1$, $F_{n+2} = F_n + F_{n+1}$ for all $n \in \mathbb{N}_0$.

   Show that for all $n \in \mathbb{N}_0$: $F_n \leq 2^n$.

   Observe that F_{n+2} is constructed from F_n and F_{n+1}: so the induction claim should cover at least F_n and F_{n+1} – unless $n = 0$ or $n = 1$ as these are define explicitly.

   (You can also try to show that $F_n = \frac{\Phi^n - (-\Phi)^{-n}}{\sqrt{5}}$ for $\Phi = \frac{1+\sqrt{5}}{2}$ – this is essentially the same proof, only the expressions get more complicated to rewrite.)

In the above tasks the identity to prove by means of induction was already given.

Typically, problem is that we are given some data (Latin: "that which is/was given"), e.g. by means of a series following some complicated expression, and then try to come up with a better (simpler, easier to evaluate, more efficient, ...) way of describing the data.

For instance, starting from the description $\sum_{i=0}^{n} i$ we actually would first compute the first few values of the resulting sequence:

$$0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, \ldots$$

and then come up with the claim that the same sequence can be obtained by evaluating $\frac{n(n+1)}{2}$. We then use induction to show our claim (as done above).

Here are some more classical series for which we can obtain simpler expressions:

4. Sum of odd natural numbers: $\sum_{i=0}^{n}(2 \cdot i + 1)$

   Remark: Using $\sum_{i=0}^{n} i = \frac{n(n+1)}{2}$ there is direct way of obtaining a simpler expression for this sum.

5. Sum of cubes: $\sum_{i=0}^{n} i^3$.

   Remark: You can also again use the already proven identity $\sum_{i=0}^{n} i = \frac{n(n+1)}{2} =: T(n)$ to avoid induction. Show that $T(n) + T(n-1) = n^2$ and $T(n) - T(n-1) = n$. Hence, $\sum_{i=0}^{n} i^3 = \sum_{i=1}^{n}(T(i) + T(i-1))(T(i) - T(i-1))$.

6. Sum of Fibonacci numbers: $\sum_{i=0}^{n} F_i$

Use e.g. a computer to compute the first elements of the corresponding sequence if you let n vary e.g. from 0 to 10.

Then try to come up with simpler way of obtaining the same numbers.

Finally, prove your claim using induction.

Nice to know: Sequences and series not only play an important role in number theory, but also arise naturally when studying e.g. the runtime of algorithms (and thus there is often connection between the two.) When given a sequence resp. the first ten elements of the sequence, it is usually a good idea to go to `https://oeis.org/` and check if a sequence with the same initial values

is known. (N.B.: only because two sequence coincide on a finite number of entries, does not prove that the two sequences coincide!)